Memory Allocators

Memory Allocators in Paged Systems

From the applications' point of view, you allocate arbitrary amounts of memory,
e.g., `malloc(23)` *allocates 23 bytes of memory*

However, the kernel only manipulates pages, not arbitrary amounts of memory!

We need two levels of allocators:

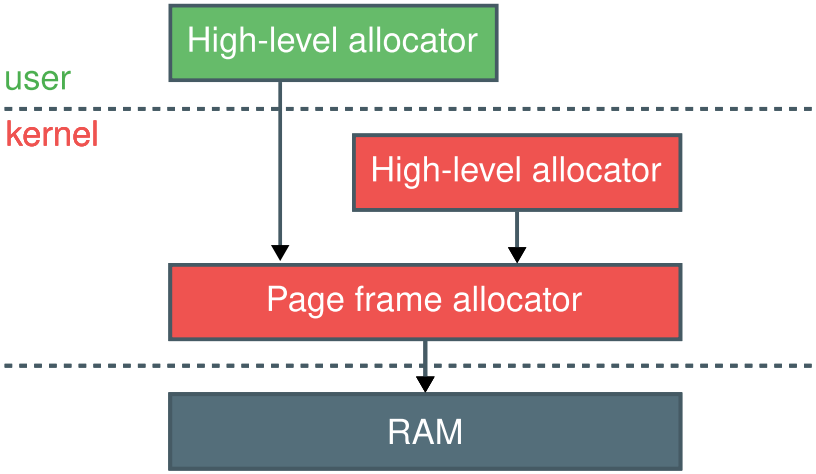
- A low-level page frame allocator in the kernel
- A high-level allocator

Memory Allocator Hierarchy

The **page frame allocator** allocates page frames in physical memory to map them with virtual pages.
It can only allocate full pages.

The **kernel high-level allocator** allocates objects of arbitrary size for use in the kernel.
Theses objects are allocated inside pages given by the page frame allocator.

The **user high-level allocator** allocates objects of arbitrary size for user programs.
It asks the kernel for memory (through system calls), and allocates objects inside this memory.



Page Frame Allocation in Linux: The Buddy Allocator

Linux uses the **buddy allocator** to manage physical page frame allocation.

Initially, physical memory is seen as a single large chunk containing all page frames, *e.g.*, $2^8 = 256$ *pages.*

Allocation:

1. The allocation size is rounded to the next power of 2
2. A chunk is repeatedly split in half until it reaches the allocation size
   The resulting half chunks are called buddies
3. The resulting chunk is allocated

If a chunk of the correct size is available, it is directly chosen

Free:

1. The freed chunk is tagged as free
2. If two buddies are free, they are merged back into a larger chunk

If two neighbors are free but not buddies, they cannot be merged!

User Space Allocator

Most user space allocators use a similar design:

- They allocate large chunks of memory at once, *e.g., multiple pages*
- Whenever a program makes an allocation, the allocator returns a pointer to a portion of the pre-allocated chunk
- They add metadata and manage the allocation and freeing of the objects without relying on the kernel, except to allocate the large chunks

Example:

A program makes a first allocation:

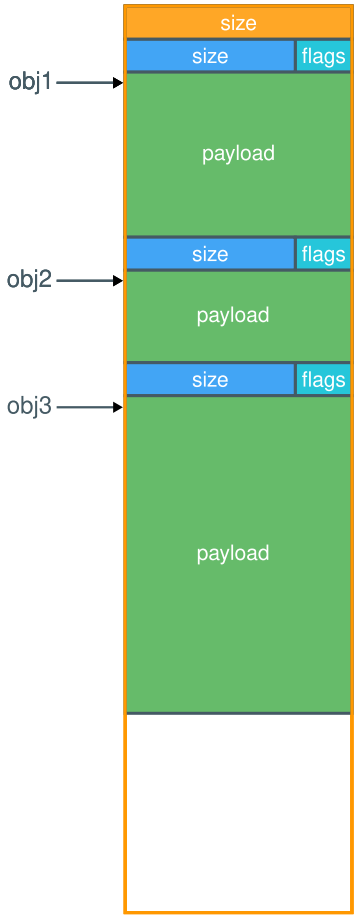
```
struct foo *obj1 = malloc(sizeof(struct foo));
```

The allocator will request a **larger piece of memory** from the kernel

It then splits this memory into a smaller object, with metadata, *e.g., size*,
and returns the address of the start of the object `obj1`

If the next allocation fits inside the remaining pre-allocated memory, the allocator continues filling the region

```
struct bar *obj2 = malloc(sizeof(struct bar));
struct fizz *obj3 = malloc(sizeof(struct fizz));
```



> Now, what happens when programs call `free()`?

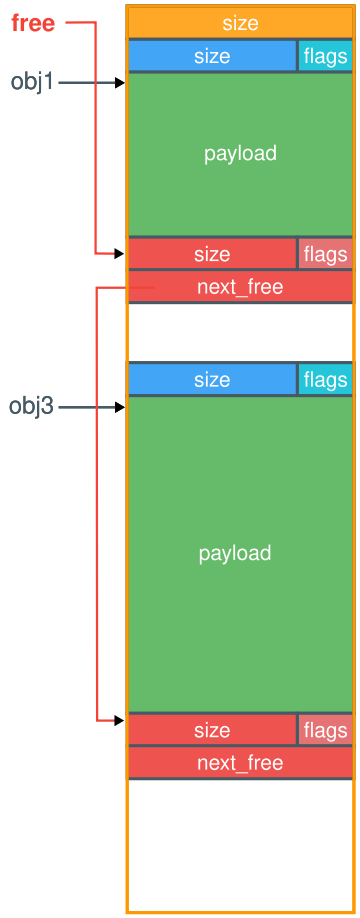
User Space Allocator (2)

The allocator needs to manage the **free space** inside the large chunks of allocated memory

They usually store a list of free memory chunks, *i.e., the holes in the pre-allocated memory*

Let's take our previous example!

1. Our program frees `obj2`, for example by calling `free(obj2);`
2. The allocator deletes the object's metadata (and potentially scrubs the data for security reasons)
3. The allocator then uses the free space to maintain a **free list**



> Managing free objects allows the allocator to avoid making system calls for every allocation!

> While most allocators use this general design idea, they vary in which data structures they use to maintain allocate/free memory, how the pre-allocate, etc.
> This has a massive impact on the performance of such allocators.

Live Q&A